

Final Project Writeup

Aaron Perrotta

Introduction

The overall goal of my project was to implement a networked FUSE filesystem for the course Linux container. By running a TCP client as a [FUSE](#) driver, the kernel sends filesystem operations to that client, which then forwards them to a remote TCP server via a custom protocol I designed and implemented. In addition to this, I wanted to better understand Unix filesystem semantics and how to design a networked protocol. With my implementation, I achieved these goals and was able to successfully mount my client as a filesystem and execute CRUD operations on such files as normal.

Design & Implementation

AZPFS is a networked filesystem built on Linux's FUSE interface. It lets one mount a remote directory as if it were local-- any file operation you perform on the mount point is transparently forwarded over TCP to a server process, which executes it against a real directory on disk and returns the result. The system is written in Rust using *Tokio* for async I/O and the *fuser* crate for FUSE integration.

There are two processes: a client daemon *azpfsd* and a server *azpfs-server*. The server is given a root directory and a TCP address to listen on. For each incoming connection it spawns a handler that reads requests, executes them against the local filesystem, and sends responses back. The client is given a mount point and the server's address. It connects over TCP and registers itself with the kernel as a FUSE filesystem. Afterwards, any filesystem operation that touches the mount point is intercepted by the kernel and delivered to the client daemon, which translates it into a protocol message, sends it to the server, waits for a reply, and returns the result back to the kernel. From the perspective of a user or program, the mount point looks and behaves like an ordinary directory.

Network Protocol

The TCP protocol follows a simple asynchronous request-response-based system. The client opens a TCP connection with the server, conducts a brief initialization handshake, and then is free to send requests to which the server responds. Importantly, the async nature of the protocol means that longer operations (like reading a large file from disk) don't block up shorter operations (like getting metadata).

Requests and responses are encoded as unique message types. These messages cover basic filesystem CRUD operations. Every message includes a "request ID" field that is assigned

uniquely by the client and re-used by the server for responses to that request, allowing the client to match them back up. A list of message types and their semantics can be found in the **Protocol.docx** file in the repo.

FsBackend Trait

The central abstraction across client and server is the *FsBackend* trait which declares an async method for each filesystem operation. The two implementations of this trait are *ClientHandler* on the client, which implements it by passing operations to the server over the protocol, and *DiskFs* on the server, which implements it by conducting those operations on the actual filesystem from a given root directory. Because of this architecture, one could easily rewrite the server's backend to instead by another *ClientHandler* repeating messages to *another* server, or rewrite the client to go straight to disk, or something else.

Client & Server Concurrency

All network and filesystem code (that is, code that is liable to block) is async. When the user does a file operation, a new task is spawned on the client that sends the relevant request and blocks for a response. A *receive_loop* task constantly runs per-client blocking on incoming responses and dispatching them to their requesting tasks via Tokio channels. When a response comes in, the receiver loop matches it via its ID, sends it over the correct channel, and thus wakes up the original requesting task. The task then returns the result back to FUSE.

On the server, a master loop accepts incoming TCP connects and spawns a new task to a dedicated *handle_client* entrypoint when one comes in. This method spawns a main loop that reads incoming messages and spawns a new Tokio task for each one, meaning multiple requests from a single client are handled concurrently. *handle_client* also spawns a sending loop that drains a channel of outgoing messages that is written to be the request handler tasks. The filesystem backend itself (*DiskFs*) is wrapped in *Arc<Mutex<>>* so that it can be shared across the per-message tasks while remaining safe to access from multiple threads.

Testing

The client and server-side code is generic not only over the filesystem backend, as discussed above, but also over the actual medium of communication between client and server. This means that tests can just use an in-memory channel, linking a *ClientHandler* to a *handle_client* task, without any real TCP or FUSE machinery or spawning multiple processes. Protocol messages are still serialized and deserialized by each side, but without the TCP overhead.

Discussion

Designing the architecture and protocol for this project was a lot of fun. The implementation was a bit more routine, insofar as it was just wiring up everything without much actual “business

logic.” For each filesystem operation, I needed to implement a FUSE handler, which called a client handler, which sent a message, which was received and unpacked by a server handler, which called a backing filesystem method, which actually did the operation on the server’s filesystem and returned the result. As such, the project had distinct “layers” that were each pretty simple. AI tools were especially helpful in this regard, particularly in writing the boilerplate of invoking my *ClientHandler* methods from the *fuser* FUSE functions.

Other areas where AI was a lot of help included:

- Writing tests and wiring up the harness for them as described above. These tests were great for implementing each filesystem operation, as I could simply implement one in each respective layer, make sure its tests pass, and then move to the next.
- Ingesting my protocol document and writing Rust structs for each message type, including annotations for de/serializing such messages over the wire via the *binrw* crate.
- Writing a custom Wireshark plugin to parse my protocol. This was a lot of fun to get working, and was really neat in helping me see what was going on.
- General spot questions, things like “How do I cast this type into that type” or “help me fix this borrow checker error,” as opposed to agentially writing code.

The AI generally did well in these areas, as I either was directly monitoring its code output (when it was writing code for core components) or because it was writing non-critical things, like the Wireshark plugin or test suite. I care a lot more about the code quality in the “business logic,” most of which I wrote myself regardless, then about the Wireshark plugin which was more of a convenience and not something I would’ve had the time to do myself.

Conclusion

Overall, I really enjoyed this project. Running it end to end for the first time was a really cool feeling, since it largely felt “invisible;” I was just using a directory as normal (which was the goal), yet I could see all the underlying protocol traffic in Wireshark enabling my use of said directory. I learned about filesystem semantics, protocol design, and got more experience working with AI effectively in a larger project, which is something I value.

Future directions I may take this include implementing more filesystem operations like file locking, improving performance via client-side caching, and making the system more robust with features like atomic transactions, or something of that nature. This is an area where I’d research deeper into existing technologies like NFS to see how these problems have been already solved in the past.